

입찰메이트 RFP 검토 지원 RAG

개발보고서 — 공공 제안요청서 98건 근거 기반 검색·답변 시스템

기간	2026. 07. 14. ~ 07. 31.
대상	공공 RFP 100건 → 고유 98건 (HWP 94 · PDF 4)
구성	박창준 · 강지연 · 구아름 · 김효진
환경	GCP · Chroma · OpenAI

01 프로젝트 개요

GCP 클라우드 환경에서 전처리 → 청킹 → 임베딩 → 검색 → 생성으로 이어지는 RAG 파이프라인을 구축했다. 먼저 빠르게 동작하는 **Naive RAG**를 기준으로 만든 뒤, 같은 데이터와 평가 질문으로 **Advanced RAG**가 얼마나 개선하는지 비교 검증했다.

기업의 제안 담당자는 공고를 받은 뒤 참가 자격 · 제출 기한 · 평가 기준 · 필수 제출물 · 계약 위험을 제한된 시간에 확인해야 한다. 원문이 길고 같은 조건이 여러 페이지에 흩어져 누락이 잦다. 이 시스템은 원문을 근거로 답하고, 그 답을 **원문 페이지**에서 검증하게 한다.

1.1 팀 구성과 역할

담당자	역할
박창준	옵시디언 초안 구성 · 전처리 · 청킹 · 벡터저장
강지연	리트리버 검색 · 검증 · PM
구아름	클라우드 관리 · LLM 생성 · 검증 · GitHub 관리
김효진	AI·기술 리서치 · 제품/UX 설계 · 앱 UI 개발

1.2 Naive RAG와 Advanced RAG

구분	Naive RAG	Advanced RAG
목적	빠르게 동작하는 기준선 확보	기준선의 검색·답변 품질 개선
청킹	Recursive split	KSS 문장 분리
텍스트 처리	구조 블록의 일반 텍스트·표 평문	같은 구조 블록 + KSS · KiwiPiePy
임베딩	text-embedding-3-small	동일
Vector DB	Chroma	동일
검색	LangChain similarity search	Hybrid + reranking + query rewriting
생성	gpt-5-nano	gpt-5-mini
평가	없음	Recall@K·MRR · RAGAS · LLM Judge

1.3 청킹 실험 계획

청킹은 Advanced RAG 품질 개선의 출발점이다. 총 5단계로 설계하고 순차적으로 진행했다.

구분	분리 방식	청크 크기	오버랩
Naive 기준선	Recursive split	512토큰	20%
Naive 기준선	라이브러리 없이 직접 split	512토큰	20%
Advanced 1차	KSS 문장 분리 + KiwiPiePy	512토큰	경계 10%
Advanced 2차	KSS 문장 분리 + KiwiPiePy	1,024토큰	경계 10%
Advanced 3차	Semantic chunking	256~1,024토큰	없음

1.4 Naive 대비 변경점

- Recursive split → **KSS 문장 분리**로 문장 경계를 보존
- KiwiPiePy** 형태소 토큰화를 더해 한국어 처리와 BM25 키워드 검색 보강
- 단순 벡터 유사도 → **Hybrid search**로 확장
- 검색 후보 **reranking** + 필요 시 **query rewriting**
- 검색은 Recall@K·MRR, 답변은 RAGAS·LLM Judge로 **이원화된 평가 체계** 적용

02 EDA — 데이터 품질 검증

전처리에 앞서 원본 100건(HWP 96 · PDF 4)의 품질을 확인했다. 여기서 나온 발견이 이후 전처리 설계를 결정했다.

항목	발견 내용	처리 결과
문서 규모	HWP는 표 미포함 시 평균 22,476자. 7건 샘플 검증 결과 표 포함 시 평균 2.68배 증가(표 개수와 상관계수 0.886)	실제 평균 약 6만자로 추정 — 표 포함 추출 방식 필요성 확인
PDF 품질	4건 모두 스캔본 아님. 페이지당 표 밀도 높음(297쪽 문서에 표 420개)	구조화된 표 추출(pdfplumber) 별도 검토
파싱 실패	HWP 1건에서 구버전 스타일로 ParseError	HWPX로 재변환하여 해결
중복 파일	크기·글자수가 완전히 일치하는 2쌍(4건)	나라장터 원본 대조로 대표 확정, 2건 제외
메타데이터 결측	<code>data_list.csv</code> 100건 중 33%(33건) 결측	나라장터·원본 대조로 17건 보정
메타데이터 신뢰도	‘사업요약’은 LLM 생성 추정 템플릿, ‘텍스트’는 본문 대비 약 29% 만 담긴 미리보기	두 컬럼 미사용 — 원본 파일 재추출로 전환

가장 큰 발견은 표였다. HWP를 표 없이 뽑으면 평균 22,476자인데 표를 포함하면 2.68배로 늘어난다. 표 개수와의 상관 계수가 0.886이다. **표를 빼면 문서의 3분의 2를 버리는 셈**이라 표 포함 추출이 필수였다.

03 데이터 파이프라인 — 박창준

HWP·PDF 원본을 표 구조를 지닌 채 검색 가능한 조각으로 만들고 벡터 DB에 넣는 단계다.

3.1 산출물

구분	산출물	규모
Naive 기준선	청크 · Chroma 1벌	31,627개 (토큰 512/102)
Advanced 전처리	구조 블록	문서 98건 · 블록 94,530개
	표 · 이미지	표 12,487개 · 이미지 886개
Advanced 청킹	3가지 조건	18,258 / 15,483 / 15,985개
인덱스	Dense + BM25	총 4벌 (Naive 1 + Advanced 3)

3.2 선택과 근거

파서를 rhwp 하나로 통일

여러 라이브러리로 나눠 읽으면 같은 문서가 파서에 따라 다르게 나온다. 텍스트만 뽑는 함수 대신 구조를 그대로 돌려주는 함수로 문단과 표를 형태째 받았다. 파싱에 실패한 HWP 1건은 복구본에서 HWPX를 만들어 다시 읽어 **98/98** 성공했다.

표를 두 형태로 동시에 보존

형태	용도	이유
Markdown	Dense 임베딩	사람이 읽는 문장에 가까워야 의미 검색이 된다
HTML	구조 보존	병합 셀까지 살려 답변에 표를 그대로 보여줄 수 있다

BM25는 표와 텍스트가 다른 필드를 쓴다. 처음엔 표를 BM25에서 제외했는데, 표에만 있는 사업명·기관명을 어휘 검색으로 못 잡아 **파이프 기호를 없앤 평문**을 따로 만들어 넣었다.

임베딩 상한(8,191토큰)을 넘는 표는 나눠야 했다. 행 묶음 단위로 자르고 쪼갠 조각에 헤더를 다시 붙였다. **세 조건 공통으로 약 95%의 표는 통째로 들어가고 나머지 5%만 나뉜다**. 어느 조건에서도 **원본 격자는 100% 보존**된다.

세 조건이 같은 문장 경계를 공유

다른 것은 **자르는 규칙 하나뿐**이다. 그래야 성능 차이를 “청킹 방식의 차이”라고 말할 수 있다. 문장 경계를 계산하는 코드를 한 곳으로 모아 세 조건이 같은 함수를 지나가게 했다.

HWP 페이지를 추정하지 않음

PDF는 실제 페이지가 있지만 HWP에는 페이지 개념이 없다. 줄 높이로 추정할 수도 있었으나 **틀린 페이지를 인용하는 것이 없는 것보다 나쁘다**고 판단해 페이지를 비우고 섹션·문단으로 표시한다. 세 조건 전부 예외가 없다.

3.3 Naive에서 발견해 Advanced에서 고친 것

구분	발견 내용	개선 방향
embeddings	document_id 와 source_id 에 같은 값이 저장됨	키 하나로 통일
embeddings	source_filename 과 file_nm 에 같은 값이 저장됨	키 하나로 통일
embeddings	[문서]/[위치]/[유형] , - 1 - 같은 키워드가 청크마다 반복	임베딩 본문에서 제외
preprocessing	고려대학교 차세대 포털 사업 PDF의 표 추출 오류	표 추출 로직 점검·수정

3.4 청킹 과정에서 고친 문제 세 가지

숫자를 만드는 것보다 틀린 것을 찾아 고치는 데 시간이 더 들었다. 셋 다 “돌아가긴 하는데 결과가 이상한” 종류였다.

01 짧은 청크가 검색 상위를 뒤덮어 표가 안 잡혔다

원인 문단마다 청크를 만들어 평균 45.7토큰짜리가 41,830개 생겼다. 짧은 청크는 벡터가 특정 단어에 쏠려 유사도가 쉽게 높아진다.

조치 문단이 아니라 섹션 단위로 문장을 채워 담도록 바꿨다.

표가 잡힌 비율 1 / 40 → 28 / 40 텍스트 청크 수 41,830 → 5,830

02 표 안 참조 ID가 임베딩 본문에 섞여 있었다

원인 HWP는 표 안에 표를 넣는 관행이 흔해 부모 셀에 포인터만 남겼는데, 그 포인터가 임베딩 본문에 남아 벡터와 어휘 색인을 오염시켰다.

조치 참조를 본문에서 빼고 청크의 부가정보 칸으로 옮겼다.

셀 마커 554 → 0 제목의 표 ID 733 → 0 이미지 참조 88 → 0

03 오버랩이 경계의 71%에서 만들어지지 않았다

원인 오버랩을 온전한 문장 단위로만 만들었는데 RFP 문장은 중앙값 96토큰이다. 51토큰 예산에 들어가는 문장이 애초에 거의 없었다.

조치 문장으로 예산을 못 채울 때 한글이 깨지지 않는 토큰 경계에서 자르도록 했다.

오버랩 있는 경계 1,247 (28.7%) → 4,640 (99.7%) 오버랩 중앙값 0 → 51

3.5 세 조건 비교

조건	설정	전체 청크	텍스트	표	평균 토큰	오버랩
케이스 1	512 / 51	18,258	5,830	12,428	561	36.2
케이스 2	1024 / 102	15,483	3,098	12,385	658	49.0
케이스 3	시멘틱 p83	15,985	3,600	12,385	627	0

표 청크는 세 조건이 거의 같다. 따라서 이 비교는 **사실상 텍스트 청크 실험**이고, 변화는 대부분 HWP에서 나온다 — PDF 페이지는 중앙값 229토큰이라 이미 66%가 512에 들어가지만 HWP 장은 평균 2,273토큰이다.

시멘틱 임계값 **0.706**의 근거 — 문장 21,455개를 임베딩해 이웃 거리 20,698개를 잴다. 평균 0.560 · **p83 0.706(선택)** · p90 0.764 · p95 0.820. 분포가 **균일한 것 자체가 발견이었다**. 화제 전환이 뚜렷한 글이라면 양극화가 나와야 하는데 이웃 문장이 기본적으로 0.56만큼 떨어져 있다. RFP가 개조식 요구사항 목록이라 문장마다 주제가 이미 다르기 때문이다.

3.6 추가 실험 — 이미지 안에 표가 숨어 있었다

이미지를 색인에서 빼도 되는지 확인하려다 **사업예산·사업기간이 적힌 표가 그림으로 붙어** 검색에서 완전히 빠져 있다는 것을 발견했다. 이미지 1,006개를 분류해 대상 424개를 확정하고 OCR 엔진 6종을 같은 표본 20장에 돌렸다.

엔진	핵심어 23개	어려운 값 8개	비용
Tesseract	21/23	6/8	무료
EasyOCR (GPU)	21/23	7/8	무료
ko-trocr	22/23	6/8	무료
gpt-5-nano	21/23	7/8	9원
gpt-5-mini	23/23	8/8	42원

gpt-5-mini 가 단독 만점이고 무료 조합(EasyOCR U ko-trocr)도 23/23이다. 다만 **실패한 6장은 여섯 엔진 모두 동일**했다 — 잉크 비율 2% 미만 빈 페이지나 가로세로비 12:1 띠 배너로 읽을 문장이 애초에 없다. 팀 결정에 따라 **파이프라인 미포함, 실험 기록**으로 남겼다.

다음 단계로 — 이 단계에서 만든 **인덱스 4벌**을 검색 파트가 받아, 설정만 바꿔 세 조건을 나란히 비교했다.

04 검색 — 강지연

벡터 유사도 단일 검색에서 **하이브리드 + 리랭킹**으로 넘어가며, 골든셋 **150문항**(1레벨 기억 · 2레벨 이해 · 3레벨 적용 각 50)으로 실측 비교했다. MAX_K=20 기준.

4.1 검색 구조

구분	Naive RAG	Advanced RAG
검색 방식	벡터 유사도 단일	하이브리드 (BM25 + 벡터)
한국어 처리	없음	kiwipiepy 형태소 토큰화
재정렬	없음	리랭킹 (Qwen3-Reranker-0.6B)
검색 단계	1단계 (top-5)	2단계 (넓게 뽑기 → 정밀 재정렬)

지표	의미
Hit@5	상위 5개 안에 정답 청크가 하나라도 있었던 질문의 비율
Recall@5	정답 청크가 여러 개일 때 그중 몇 %를 상위 5개에서 찾아냈는가
MRR@5	정답이 상위 몇 번째였는지 점수화 (1등 1점 · 5등 0.2점)
Coverage@5	상위 5개가 답변에 필요한 정보를 얼마나 폭넓게 담았는지

4.2 청크 조건별 성능

동일한 하이브리드+리랭킹 파이프라인에서 **청크 버전만** 바꿔 비교했다.

청크	단계	Hit@5	Recall@5	MRR@5	Cov@5	Hit@10	Recall@10	Cov@10
V512	Hybrid	0.4333	0.3289	0.3780	0.5045	0.4533	0.3456	0.5649
V512	Rerank	0.6733	0.5322	0.5372	0.6723	0.7133	0.5889	0.7390
V1024	Hybrid	0.4067	0.3478	0.3410	0.5031	0.4267	0.3611	0.5593
V1024	Rerank	0.6800	0.5967	0.5354	0.7025	0.7400	0.6467	0.7622
VSEM	Hybrid	0.3733	0.2783	0.3319	0.4302	0.4200	0.3106	0.5059
VSEM	Rerank	0.6533	0.5033	0.5214	0.6338	0.7133	0.5689	0.7031

두 가지가 드러났다. 첫째, 리랭킹이 결정적이다 — 세 조건 모두 Recall@5가 **20%p** 내외 된다. 둘째, 청크 조건 중에서는 **V1024가 가장 높다**(Recall@5 0.5967). 이후 실험은 모두 V1024를 기준으로 진행했다.

4.3 하이브리드 가중치

가중치 조합	Hybrid Recall@5	Rerank Recall@5	Rerank MRR@5	Rerank Cov@5
w37 (BM25 0.3 : 벡터 0.7)	0.3478	0.5967	0.5354	0.7025
w55 (BM25 0.5 : 벡터 0.5)	0.4967	0.6033	0.5368	0.7080
w73 (BM25 0.7 : 벡터 0.3)	0.5367	0.6033	0.5368	0.7080

리랭킹 **이전**에는 가중치를 올릴수록 Recall@5가 뚜렷하게 향상됐다(0.3478 → 0.5367). 그러나 리랭킹 **이후**에는 w55와 w73이 사실상 같아지고 w37과의 차이도 크게 줄었다. 즉 **리랭킹이 1차 후보군의 품질 차이를 상당 부분 흡수한다**.

4.4 리랭킹 후보군 — 성능과 지연시간의 교환

후보군	Recall@5	Recall@10	Recall@20	평균 지연
후보 10개 (채택)	0.6033	0.6467	0.6600	2.39초
후보 25개 (확장)	0.5944	0.6800	0.7300	4.40초

k=5에서는 차이가 거의 없지만 k가 커질수록 후보군을 넓힌 쪽이 유리하다(Recall@20 0.66 → 0.73). 다만 평균 지연이 **2.39초에서 4.40초로 약 1.8배** 늘어난다. 응답 속도를 우선해 **후보 10개로** 결정했다.

4.5 난이도별 성능

난이도	문항	Hit@5	Recall@5	MRR@5	Coverage@5
1레벨 · 기억	50	0.68	0.65	0.4963	0.7068
2레벨 · 이해	50	0.70	0.63	0.6100	0.7528
3레벨 · 적용	50	0.68	0.53	0.5040	0.6645

여러 정보를 종합해야 하는 **3레벨 적용**에서 **Recall@5가 0.53**으로 낮다. 실제로 3레벨의 평균 정답청크수는 1.64개로 1레벨(1.16개)보다 많고 첫 정답 순위도 1.88위로 밀려 있다. 여러 근거를 동시에 모아야 하는 질문일수록 검색이 어려워진다.

4.6 검색 최종 채택 구성

항목	채택값
청크	V1024 — 1,024토큰 · 오버랩 102
1차 검색	하이브리드 · 가중치 w73 (BM25 0.7 : 벡터 0.3)
후보군	BM25 10 + 벡터 10 → 최대 20개 풀
리랭커	Qwen3-Reranker-0.6B
최종 성능	Recall@10 0.6467 · MRR@10 0.5442 · Hit@10 0.7400 · Coverage@10 0.7622
평균 지연	2.39초

후보군을 25개로 넓히면 Recall@10이 0.6800까지 오르지만 지연이 4.40초로 늘어난다. 사용자가 화면에서 기다리는 시간을 우선해 후보 10개 구성을 최종 결정했다.

다음 단계로 — 검색이 가져온 문맥을 답변이 얼마나 충실히 반영하는지가 다음 절의 Faithfulness다.

05 생성 · 검증 — 구아름

GCP 공용 개발 환경을 구축하고, 검색 결과로 답을 만드는 **Generate** 모듈과 품질을 재는 **Evaluation** 모듈을 구현했다.

5.1 산출물

구분	산출물
개발 환경	공용 GCP VM · 팀원 계정 및 SSH · VSCode Remote 연동
협업 체계	GitHub 저장소 · lint 설정 · Discord Webhook
Generate	BidMateRAGSession 기반 답변 생성 엔진
Evaluation	RAGAS 정량 평가 · LLM Judge 정성 평가
평가 결과	CSV 및 DataFrame 출력

5.2 개발 환경과 협업 체계

프로젝트 초기에 가장 먼저 한 일은 **GCP에 공용 VM을 세우는 것**이었다. 개발 초기부터 배포에 가까운 공통 작업 환경을 마련해 이후 기능 개발과 테스트를 안정적으로 진행할 수 있었다.

공용 VM으로 환경을 통일

단일 로컬이 아닌 공용 VM 기반으로 작업 공간을 구성해 팀원 간 **의존성·경로 문제를 사전에 줄였다**. 팀원별 계정을 만들고 권한을 나눠 안전하게 접속하도록 했다.

VSCode Remote 연동

원격 서버에서 로컬 IDE처럼 **파일 편집·실행·디버깅**이 가능하다. 웹 콘솔이나 단순 터미널에 의존하지 않아 익숙한 도구로 협업할 수 있었고, 실행 환경 통일로 **디버깅 시간이 줄었다**.

lint로 스타일을 자동 관리

각자 다른 스타일로 쓰면 가독성이 떨어지고 유지보수가 어려워진다. 자동화로 관리해 코드 리뷰에서 **사소한 스타일보다 실제 로직에 집중**할 수 있었다.

Discord Webhook 연동

저장소의 커밋·푸시·PR 변경을 팀 채널로 즉시 공유해 **커뮤니케이션 지연을 줄였다**. 단순 소스 관리를 넘어 협업 상태를 실시간 공유하는 운영 기반 역할을 했다.

5.3 Generate 모듈 — 어떻게 만들었나

BidMateRAGSession을 중심으로 한 답변 생성 모듈이다. 질문만 모델에 넘기는 방식이 아니라 **검색된 문맥을 함께 반영**해 실제 RAG 흐름과 동일하게 동작하도록 설계했다.

검색과 생성의 분리

사용자 질문이 들어오면 먼저 **Query Rewrite**로 후속 질문이나 문맥 의존 표현을 검색용 독립 질의로 바꾼다. 그 질의로 retriever가 문서를 찾고, 검색 결과는 그대로 쌓지 않고 **중복 체크와 중복 텍스트를 제거**한 뒤 하나의 압축된 문맥 문자열로 정리한다. 이때 출처·페이지·체크 ID·점수를 함께 넣어 근거성은 지키면서 토큰 낭비는 줄였다.

비동기 처리와 응답 형식 고정

OpenAI **Async Responses API**로 모델 호출을 비동기 처리해 전체 파이프라인의 응답성을 높였다. 여기에 **Structured Output(JSON Schema)**을 적용해 모델이 정해진 형식만 반환하도록 제한했다. 그 결과 답변·요약·근거·확신도·추가확인필요 같은 필드를 안정적으로 관리할 수 있고, 생성 결과를 후속 평가에 바로 넘길 수 있다.

멀티턴에서 토큰을 아끼는 방법

대화 상태는 서버 측 응답 ID로 이어가고, **최근 대화만** 짧게 보관한다. 긴 이력을 매번 다시 넣는 대신 **대화 요약과 수집된 핵심 필드**를 따로 관리한다. 맥락은 보존하면서 입력 길이를 줄여 장기 대화에서도 안정적으로 동작한다.

응답이 깨질 때의 복구

모델 출력에서 유효한 JSON만 골라내고, 파싱에 실패하면 **재시도 로직**이 동작한다. 문맥과 대화 이력이 과도하게 길어지지 않도록 문맥 길이·문서 개수·최근 메시지 수에 **상한**을 뒀다.

정리하면 Generate 모듈은 **검색과 생성을 분리하면서도 실제 RAG 흐름을 유지**하고, 멀티턴과 후속 질문을 고려한 세션 기반 생성 엔진이다. 비동기 처리 · Query Rewrite · 문맥 압축 · 토큰 예산 관리 · JSON 복구를 결합해 **생성 안정성과 응답 일관성**을 확보했고, 이후 RAGAS와 LLM Judge 평가를 수행할 기반을 만들었다.

5.4 Evaluation 모듈 — 두 갈래로 나눈 이유

RAG 성능을 다각도로 검증하기 위해 **정량**과 **정성** 평가를 함께 구현했다.

실제 운영 흐름을 그대로 평가

두 평가 모두 `session.ask()` 만 부르지 않고 **질의 재작성 → 검색 → 생성** 순서를 실제와 똑같이 거친 뒤 결과를 평가한다. 그래야 측정값이 실사용 성능을 대변한다.

RAGAS 정량 평가

평가용 데이터를 읽어 질문마다 실제 RAG를 실행하고, 그 결과를 RAGAS 입력 형식으로 바꿔 네 지표를 계산한다. **검색 결과의 실제 본문**을 문맥 필드에 넣어야 검색 기반 지표가 의미 있게 계산된다. 질문별 재작성·검색·생성 시간과 전체 소요를 함께 기록하고 CSV로 저장해 재현 가능하게 했다.

LLM Judge 정성 평가

RAGAS만으로는 확인하기 어려운 실무 품질을 **루브릭 기반**으로 보완한다. 질문·기준 정답·생성 답변·검색 문맥을 심사 모델에 넘겨 다섯 항목에 1~5점을 매기고, 종합 점수와 판정(pass/borderline/fail)을 산출한다. 여기에 **판단 이유·빠진 논점·근거 없는 주장**을 남겨 점수뿐 아니라 왜 부족한지도 분석할 수 있다.

평가 안정성

심사 입력도 검색 문서의 **실제 본문만** 추출해 쓰고, 질문·답변·정답·문맥 길이를 제한해 프롬프트가 과도하게 길어지지 않게 했다. 일부 질의가 실패해도 **전체 결과를 최대한 CSV로 남기도록** 예외 처리를 넣었다.

RAGAS로 **정량 추적**을, LLM Judge로 **정성 심사**를 함께 구성해 파이프라인 품질을 입체적으로 검증할 수 있는 평가 체계를 만들었다.

5.5 고친 문제 세 가지

문제	원인	조치	결과
팀원별 개발 환경 차이	실행 환경·라이브러리 버전 차이로 같은 코드가 다르게 동작	GCP VM + SSH + VSCode Remote 구성	동일 서버에서 개발·테스트
생성 응답 형식 불안정	모델이 일정하지 않은 출력 형식을 반환해 JSON 파싱 오류	Structured Output(JSON Schema) + 복구 로직	후처리·평가에 바로 활용
성능 평가의 한계	생성 답변만으로는 검색·답변 품질을 객관적으로 판단 불가	RAGAS + LLM Judge 병행	정량·정성 평가 체계 구축

5.6 평가 결과 — RAGAS (n=150)

지표	평균	의미
Answer Relevancy	0.609	답변이 질문 의도에서 벗어나지 않았는지
Faithfulness	0.712	답변이 검색된 문서에 실제로 근거하는지 (환각 여부)
Context Precision	0.863	검색된 문서 중 실제로 답변에 쓸모 있었던 비율
Context Recall	0.747	정답에 필요한 정보가 검색 결과에 빠짐없이 담겼는지

평가 조건에 주의가 필요하다. 비용·시간 제약으로 **k=2 · context 1,200자 · response 800자**로 축소해 측정했다. 프로덕션 설정(k=10, 전체 길이)보다 낮게 나왔을 수 있어 절대 수치보다 **케이스 간 상대 비교** 위주로 해석해야 한다.

5.7 평가 결과 — LLM Judge (gpt-5-mini, n=150)

항목	5점 만점	판정	건수	비율
Correctness · 정확성	3.95	Pass	112	74.7%
Groundedness · 근거 충실도	4.15	Borderline	20	13.3%
Relevance · 질문 적합성	4.49	Fail	17	11.3%
Completeness · 완전성	4.01	Judge Error	1	0.7%
Clarity · 명확성	4.44			

Overall Score **4.20 / 5.0**이고 150건 중 **112건(74.7%)**이 Pass다. 항목별로는 Relevance(4.49)와 Clarity(4.44)가 높고 **Correctness(3.95)**가 가장 낮다 — 읽기 좋고 질문에 맞는 답을 하지만 사실 정확도에서 아쉬움이 남는다는 뜻이다.

5.8 한계

- Query Rewrite와 토큰 제한 규칙이 **휴리스틱 기반**으로 동작한다. 추가 최적화가 필요하다.
- LLM Judge는 **모델 기반 평가**이므로 결과의 일관성을 높이는 검증이 필요하다.
- 다양한 입찰 문서와 실제 사용자 질문을 활용한 **대규모 성능 평가**가 남아 있다.

06 리서치·앱 UI — 김효진

평가 기준과 파이프라인 차이를 근거로 정리하고, 이를 사용자가 **RFP를 검토·판단·실행하는 흐름**의 Copilot UI로 연결했다.

6.1 Golden Set — 전체 확장보다 작은 Pilot 우선

98개 RFP 전체를 곧바로 정답셋으로 만들면 잘못된 질문 유형이나 근거 매핑 오류가 대량으로 반복된다. 먼저 **대표 10문서·100문항 Pilot**을 설계했다. **체크 ID는 파서가 바뀌면 달라지므로** 문서·섹션·필수 사실을 안정 기준으로 두고 체크 ID는 추 적용 보조로만 썼다.

6.2 파이프라인 비교 — 성능 숫자보다 계약 차이

두 저장소를 “구버전/신버전”으로 단순 비교하지 않고 Git 시점·파일 내용·corpus·청킹·인덱스를 함께 확인했다. 그 결과 Main Advanced RAG가 독립 신규 파이프라인이 아니라 **원본 스냅샷을 포팅한 분기판임**을 확인했다. 서로 다른 corpus의 Chroma와 BM25를 **섞지 않는 것**을 우선 원칙으로 정했다.

6.3 제품 설계 — 기술 모듈이 아니라 업무 단계에서 출발

초기 설계에서 Chunking·Embedding·Retriever 같은 기술 모듈을 화면 중심에 두지 않았다. 사용자가 RFP를 받은 뒤 **실제로 수행하는 업무를** 관찰 단위로 삼았다. 핵심 질문은 “어떤 정보를 찾는가”보다 **“그 정보로 어떤 결정을 내리는가”**였다.

사업 발견 → 핵심 내용 파악 → 적격성 판단 → 수행 가능성 검토 → 평가·위험 검토 → 참여 판단 → 제안 준비 → 원문 근거 확인

사용자 질문	실제 판단	제품 응답 방식
예산이 얼마인가?	수행 가능한 규모와 수익성인가?	예산·기간·범위 요약과 원문 근거
참가 자격은?	실격 가능성 없이 참여 가능한가?	자격별 충족·불충족·확인 필요 상태
무엇을 제출하는가?	마감 전 준비할 업무는 무엇인가?	입찰 제출물과 계약 후 산출물 분리
위험 조건은?	참여 또는 계약 시 감수할 위험은?	실격·감점·불이익을 심각도별 표시

주 사용자는 기업·기관의 제안 담당자다. 공고 접수부터 선정평가·계약·착수까지 여러 이해관계자와 정보를 공유하므로, 결과는 단순한 AI 답변이 아니라 **담당자가 확인하고 상태를 바꾸며 다음 행동으로 넘길 수 있는 작업 단위**여야 한다.

MVP 원칙

원칙	내용
중요도 우선	마감·자격·실격·제출물을 같은 비중으로 나열하지 않고 우선순위를 시각화했다.
근거 우선	모든 분석 카드는 문서명·페이지·원문으로 연결되도록 했다.
행동 연결	분석 결과를 체크리스트·상태 변경·담당 업무로 전환했다.
보수적 표현	근거가 없거나 신뢰도가 낮으면 단정하지 않고 ‘확인 필요’ 로 구분했다.

MVP 제외 범위 — 회사 내부 실적·역량 자동 매칭, 여러 RFP 자동 비교, 제안서 전체 자동 작성, 참여 여부 자동 결정과 수주 가능성 예측은 향후 확장으로 분리했다.

6.4 화면과 정보 구조

화면	목적	주요 기능
RFP List	검토할 문서 선택	업로드 · 검색 · 정렬 · 상태 · 마감일 · 진행률
Workspace	문서별 판단과 준비	Overview · Go/No-Go · 실행 준비 · 요구사항 · AI 질문
Evaluation	내부 품질 점검	모듈 상태 · 검색·생성 평가 · E2E 운영 지표

Workspace는 3열 구조다. 왼쪽에 파일 목록과 목차, 가운데에 업무 탭, 오른쪽에 원문 Evidence 패널을 뒀다. 분석 카드를 클릭해도 **화면을 이탈하지 않고** 해당 원문 페이지를 확인할 수 있다. Evidence 패널 폭은 사용자가 조절·유지할 수 있어 긴 표와 본문을 읽기 쉽다.

업무 탭	역할
Overview	제출·질의 마감, 자격, 위험, 제출물 진행률과 지금 할 일을 요약
Go / No-Go	참가 자격과 실적·감점·계약상 불이익을 확인하고 검토 상태를 저장
실행 준비	입찰 제출서류와 계약 후 산출물을 분리해 준비 상태를 관리
요구사항	기능·품질보안·수행계약 요구사항을 검색하고 원문 표로 이동
AI 질문	자유 질문에 짧은 결론과 출처를 제공하며 근거 페이지로 이동

시각 설계 — 업무용 도구의 신뢰성과 가독성을 위해 흰 카드와 연회색 배경, 네이비 상태바를 썼다. 실적은 빨강, 감점은 주황으로 구분하고 **과도한 그라데이션과 그림자는 배제했다.**

6.5 구현하며 고친 것

- Overview와 업무 탭이 서로 다른 데이터를 표시하던 문제를 통합해 **건수 불일치를 해소했다.**
- 실적·감점뿐 아니라 위약금·손해배상·계약 해지 등 **계약상 불이익까지** 위험 범위를 넓혔다.
- 위험 상태를 미확인·위험 있음·문제없음으로 나눠 **‘분석 전 0건’과 ‘문제 없음’을 구별했다.**
- 제출물 검색어를 다중화하고 빈 결과 캐시를 방지해 **누락 가능성을 낮췄다.**
- 입찰 단계 제출서류와 계약 후 수행 산출물을 분리해 **준비 시점을 명확히 했다.**

판단 상태와 실행 상태를 저장한다. 참가 자격은 충족·불충족·확인 필요로, 실행 준비는 미착수·진행 중·완료로 바꿀 수 있다. 새로고침 후에도 유지되는지 자동 테스트로 확인했다. **분석값과 사용자 판단을 분리해** AI 제안이 최종 의사결정을 대체하지 않도록 했다.

Mock과 실제 API를 전환하는 구조를 썼다. 초기 UI 개발과 회귀 테스트에는 Mock을 쓰고 통합 단계에서 환경 설정으로 실제 백엔드로 바꾼다. **외부 API 비용과 네트워크 상태에 UI 개발이 종속되는 문제를 줄였다.**

6.6 성능 최적화

- 동일 문서 분석 결과와 동일 질문 답변을 캐시해 반복 LLM 호출을 막았다.
- 프론트엔드에서 탭별 결과를 캐시해 같은 탭 재진입 시 중복 API 요청을 줄였다.
- 문서 페이지·목차·검색은 LLM을 거치지 않고 즉시 처리한다.
- API 처리시간 헤더와 백엔드 timing 로그를 추가해 병목 구간을 구분했다.

반복 사용 시 체감 응답이 약 5초에서 3초 수준으로 줄었다. 검색만 보면 warm 상태에서 약 0.1초이고 실제 답변 시간은 LLM 호출이 더해진다. 품질 손실을 피하려고 분석 탭의 검색 방식과 입력 길이를 무리하게 줄이지는 않았다.

6.7 테스트

단순 렌더링 확인을 넘어 실제 사용자의 행동 흐름을 테스트 기준으로 정의했다.

검증 영역	대표 시나리오	기대 결과
상태 저장	자격·준비 상태 변경 후 새로고침	사용자 선택 유지
문서 탐색	검색어 입력 후 결과 페이지 클릭	원문 뷰어가 해당 페이지 표시
정보 일관성	Overview와 세부 탭 건수 비교	동일 기준·동일 건수

6.8 수치

항목	실측
Golden Set v3 전체 문항	104 (답변 가능 95 / 불가 9)
멀티홉 문항	48 (46.2%)
난이도 Easy / Medium / Hard	27 / 46 / 31
v3 검수 상태	draft 102 / 섹션 매핑 확인 필요 2
파이프라인 핵심 파일 대조	8개 (일치 4 / 포팅 수정 4)
앱 업무 화면	Workspace 5개 탭 + 문서 목록 + 내부 평가

6.9 Streamlit UI 구성

콘솔 버전(main.py)의 파이프라인(질의 재작성 → 하이브리드 검색 → 답변 생성)은 그대로 두고 화면만 채팅형 웹 인터페이스로 옮겼다.

기능	설명
멀티턴 채팅	채팅창 UI로 이전 질문의 맥락을 이어서 질문
답변 표시	직접 답변 + 요약 + 신뢰도 배지, 추가 확인 안내, 문서 간 충돌 경고
근거 인용	출처 문서·페이지·chunk_id·유사도·인용 문구를 펼쳐보기로 표시
검색 문서 목록	리랭킹 후 상위 k개를 사업명·발주기관·금액·마감일과 함께 본문 미리보기
사이드바 설정	모델명 표시, 검색 문서 개수(k, 1~10) 슬라이더, 대화 초기화

07 최종 채택안과 근거

요소	채택안
청킹	1,024토큰 · 10% 오버랩 · 페이지/문단 경계 (KSS + KiwiPiePy)
검색	Hybrid Search (BM25 0.7 : 벡터 0.3)
재랭킹	Qwen3-Reranker-0.6B — 후보군 최대 20개 → candidate_k 10 → 최종 10
생성 모델	gpt-5-mini

0.647 Recall@10 검색	0.740 Hit@10 검색	0.712 Faithfulness 생성	4.20 LLM Judge / 5.0
--------------------------	-----------------------	-----------------------------	----------------------------

7.1 채택 근거

Advanced RAG 3개 청킹 케이스 중 **검색품질 기준으로 1024토큰이 가장 우수**해 생성품질 검증 대상으로 선정했다. 이 구성은 Hit@10 0.740으로 **네 질문 중 세 개꼴로** 정답 문서를 상위 10개 안에서 찾아냈고, LLM Judge Overall 4.20 / Pass율 74.7%로 실사용 가능한 수준의 답변 품질을 보였다.

Naive 대비 개선 근거 — KSS 문장분리로 문장 경계 보존, KiwiPiePy 형태소 분석으로 BM25 키워드 매칭 정확도 향상, Hybrid search + reranking으로 단순 벡터 유사도 검색의 한계(짧은 키워드 질의 취약점)를 보완했다.

단계	Naive	Advanced	근거
청킹	글자 기준 · 표 평문	문장 경계 · 표 이중 보존	표 격자 100% 보존
검색	벡터 단일 top-5	하이브리드 + 리랭킹	Recall@5 0.35 → 0.60
생성	gpt-5-nano · 단순 응답	gpt-5-mini · Rewrite + JSON Schema	Faithfulness 0.712
검증	없음	RAGAS + Judge + Evidence	Judge 4.20 / 5.0

08 결론과 한계

정성적으로 설계한 개선 방향(문장 경계 보존 · 하이브리드 검색 · 재랭킹 · 이원화 평가체계)이 실제 수치로 확인됐다. Context Precision 0.863으로 검색된 문서 대부분이 유용했고, LLM Judge Pass율 74.7%로 네 건 중 세 건은 신뢰할 만한 답변을 생성했다.

8.1 한계

항목	내용
검색 재현율	Recall@10 0.647 – 정답 문서 3건 중 1건 정도는 상위 10개 안에서도 놓친다
Answer Relevancy	0.609로 네 지표 중 가장 낮다 – 질문 의도에서 벗어난 답변이 상대적으로 많다
Faithfulness 걸쳐	150건 중 39건이 계산 불가 – 평가 파이프라인 자체의 안정성 점검 필요
Fail / Borderline	150건 중 37건(24.6%)이 완전한 합격 판정을 받지 못했다
응답 속도	질문당 평균 32.6초 – 대부분(29.6초)이 LLM 생성 단계로 실사용에는 느리다
RAGAS 평가 조건	비용·시간 절감을 위해 k=2 · context 1,200자 · response 800자 로 축소 평가. 프로덕션 설정(k=10)과 달라 절대 수치가 낮게 측정됐을 수 있다

8.2 향후 계획

- Fail 17건과 Borderline 20건의 missing_points · unsupported_claims 를 분석해 검색 실패와 생성 실패를 유형화하는 원인 분석이 선행되어야 한다.
- 검색 개선 — 임베딩 모델 교체 또는 query expansion, 리랭커를 Cohere Rerank로 교체하는 방안 검토.
- 3레벨(적용) 질문은 여러 근거를 모아야 해 Recall@5가 0.53으로 낮다. 멀티쿼리로 질문을 분해하는 방식이 필요하다.
- 생성 속도 — gpt-5-mini 대신 Gemini 계열 교체 실험, 프롬프트 경량화, 스트리밍 응답 적용.
- Rewrite 고도화로 답변 생성 정확도를 높인다.
- RAGAS를 프로덕션 설정(k=10, 전체 길이)으로 재평가해 축소 평가로 인한 저평가 여부를 검증한다.
- 이미지 속 표는 OCR로 회수 가능함을 확인했으나 파이프라인 미포함 — 로고·배너 필터를 먼저 만든다.
- Golden Set v4 Pilot 100문항의 생성·검수·split 확정이 남았다.

09 부록 — 재현 방법

9.1 환경 준비

```
git clone https://github.com/aromgu/sprint_mid_project_team1.git
cd sprint_mid_project_team1
pip install uv
uv sync
uvx pre-commit install          # 커밋 전 자동 lint/format
```

9.2 데이터 파이프라인

```
# Naive 기준선
python3 scripts/run_chunking.py
python3 scripts/run_indexing.py

# Advanced — 전처리 → 청킹(3조건) → 인덱싱
LD_PRELOAD=/lib/x86_64-linux-gnu/libfreetype.so.6 \
python3 scripts/run_advanced_preprocessing.py          # 약 5분
python3 scripts/run_advanced_chunking.py --max-tokens 512
python3 scripts/run_advanced_chunking.py --max-tokens 1024
python3 scripts/run_semantic_sentence_dump.py
python3 scripts/run_semantic_distances.py
python3 scripts/run_semantic_chunking.py
python3 scripts/run_advanced_indexing.py

python3 -B -m unittest discover -s tests
```

run_advanced_chunking.py 의 기본 입력 경로가 preprocessed_v1/ 인데 실제 데이터는 preprocessed_v4/ 에 있다. 기본값으로 실행하면 FileNotFoundError 가 나므로 --documents · --blocks 로 명시해야 한다.

9.3 생성과 평가

```
# 답변 생성
uv run python main.py

# RAGAS 정량 평가
uv run src/evaluate/eval_ragas.py

# LLM Judge 정성 평가
uv run src/evaluate/eval_llm_judge.py
```

9.4 앱 실행

```
uv run streamlit run streamlit_app.py
```

입찰메이트 RFP 검토 지원 RAG 개발보고서 · 중급 프로젝트 팀 1 · 2026. 07.

모든 수치는 실행 산출물에서 확인한 실측값이다.